Reverse-Engineering a No-Touch IR Thermometer

From EEPROM Snooping to WiFi-Controlled Readings

Introduction

Every non-contact IR thermometer looks like a sealed black box from the outside: point it at your forehead, press the button, and a number appears. But underneath, it's just a small microcontroller (MCU) reading an infrared sensor, running the value through some math, and pushing it to a display - with an EEPROM chip along for the ride to store calibration data or logs. This write-up documents the full process of reverse-engineering exactly how one of these devices works, using nothing but an ESP32, some jumper wires, and a lot of careful observation.

The goal throughout was simple: figure out what raw data the thermometer is working with internally, and how that raw data becomes the Fahrenheit number on the screen - without ever opening the manufacturer's firmware.

Tools and Setup

- Target device: a handheld IR thermometer with an onboard EEPROM (HE24C02-type) and a separate IR sensor chip, both communicating with the main MCU over I2C.
- Sniffer/controller: an ESP32 dev board, used first as an I2C slave emulator, later as a fully passive bus sniffer, and finally as a WiFi-connected controller.
- Software: Arduino IDE, Serial Monitor for live logs.
- A multimeter, for continuity-testing ground vs. signal pads before wiring anything.

Phase 1 — Logging the EEPROM

The first foothold into the device was its EEPROM. Rather than trying to intercept traffic passively (which came later), the initial approach had the ESP32 pretend to BE the EEPROM - acting as an I2C slave at the EEPROM's address, accepting every write the main MCU sent, and mirroring the full 256-byte memory image after each write. This meant nothing needed to be desoldered; the ESP32 simply sat on the same bus and impersonated the chip.

Every write showed up in the serial log as a clean frame, for example:

```
=== I2C WRITE FRAME ===
Bytes received: 3
000 : 0x26  (&)
001 : 0x10
002 : 0x0E
======================
```

followed by a full mirror dump of the simulated EEPROM contents, so the exact address being touched was always visible.

What the EEPROM logging revealed

- Address 0x26/0x27 - a 2-byte little-endian value that changed on every single reading. This was clearly measurement-related.
- Address 0x28 - a single byte that incremented by exactly 1 every time a reading was taken. This behaves exactly like a measurement counter or record index, not a temperature.
- A second region starting around 0x56, stepping by 2 bytes per reading (0x56, 0x58, 0x5A, 0x5C, 0x5E...) in lockstep with the 0x28 counter - a rolling history log storing a copy of the same raw value.

Comparing 20 real thermometer readings against their corresponding EEPROM writes made one thing obvious: the same displayed temperature (for example, 96.8 F) showed up paired with several different raw values, and different displayed temperatures sometimes shared very similar raw values. That ruled out a simple one-to-one mapping at the EEPROM

layer. The EEPROM was clearly storing something upstream of the final displayed number - a raw or intermediate measurement, not the answer itself.

Conclusion of Phase 1: the EEPROM confirmed where a measurement-related value lived and how the device indexes its history, but it could not by itself explain the display formula. The next step had to be watching the sensor bus directly, in real time, as a reading was taken.

Phase 2 — Building a Passive I2C Sniffer

The EEPROM approach worked by impersonating a chip - useful, but it meant the ESP32 was an active participant on the bus. To watch the real sensor-to-MCU conversation without disturbing it, a genuinely passive sniffer was needed: something that only listens, never drives the lines, and never even calls the standard I2C library (which would configure the pins as an active master or slave).

How the sniffer works

I2C has two lines: SCL (clock) and SDA (data). A passive sniffer only needs to watch both:

- An interrupt fires on every SCL rising edge. At that instant the state of SDA is guaranteed stable and represents one data bit, so the interrupt samples SDA and shifts it into a byte buffer.
- A second interrupt fires on every SDA change, but only pays attention to it while SCL is currently HIGH. A falling SDA edge while SCL is high is, by the I2C specification, a START condition; a rising SDA edge while SCL is high is a STOP condition. Any SDA change while SCL is LOW is just normal mid-bit data and gets ignored.
- Every 9 bits (8 data bits + 1 ACK/NACK bit) forms a complete byte, which gets pushed into a small ring buffer.
- The main loop() drains that ring buffer, reconstructing full transactions: START, address byte (with its embedded read/write flag), one or more data bytes with ACK/NACK, then STOP.

Both GPIO pins are configured as plain INPUT - never OUTPUT - so the sniffer cannot interfere with the real bus traffic, no matter what goes wrong in the code.

Wiring the sniffer

- Tap SDA and SCL in parallel with jumper wires at a convenient point between the sensor chip and the MCU - do not cut any existing trace.
- Confirm the bus voltage with a multimeter before connecting. Many small sensor breakout boards run at 3.3V and can connect directly; if the bus is 5V, a logic-level shifter or simple resistor divider is required to protect the ESP32's GPIOs.
- Common ground between the ESP32 and the thermometer board is required for any of this to work.

Phase 3 — Capturing Real Sensor Traffic and Cracking the Formula

With the passive sniffer running, taking a reading on the thermometer produced full I2C transactions in real time. The same 0x26 write pattern seen from the EEPROM side reappeared here, now directly tied to live sensor activity, for example:

```
=== FRAME WRITE addr=0x10 ===
  byte[0] = 0x26  (ACK)
  byte[1] = 0x69  (ACK)
  byte[2] = 0x0F  (ACK)
--- STOP ---
```

Reading byte[1] and byte[2] together as a little-endian 16-bit value (0x0F69 = 3945 in this example) and comparing it against the number actually shown on the thermometer's display across many readings produced the following data:

Displayed (F)	Raw hex	Raw decimal	Naive conversion (F)	Match?
96.8	0x0E10	3600	96.80	Exact
96.9	0x0E1A	3610	96.98	Close
98.0	0x0E5A	3674	98.13	Close
103.1	0x0F69	3945	103.01	Close
106.8	(0x103E)	4158	106.90	Close
109.2	0x1041	4161	106.90	Off by 2.3 F
“HI” (error)	0x7F3F / 0x7F41	32575 / 32577	n/a	Sentinel, not a reading

The pattern that emerged for the normal range of readings was clean and simple:

```
degrees_C = raw_value / 100
degrees_F = degrees_C * 9 / 5 + 32
```

This matched the display almost exactly for every reading roughly in the 96-103 F range. Two additional, very useful discoveries came out of this same dataset:

- Any raw value in the 0x7Fxx range (for example 0x7F3F) is an out-of-range / error sentinel, not a real measurement. This is exactly what causes the thermometer to display “HI” instead of a number, and must be filtered out before applying the formula.
- A 109.2 F reading broke the simple formula - it computed to only 106.9 F, a 2.3 F gap far larger than the small rounding differences seen everywhere else. A very similar raw value on a separate occasion had matched a 106.8 F display almost exactly, meaning the raw value alone does not fully determine the displayed number at the high end.

Phase 4 — The Nonlinearity Problem

The 2.3 F discrepancy at the high end is not measurement noise - every other data point lined up within a tenth of a degree, so a gap that size is a real effect. The leading explanation is that non-contact / forehead thermometers commonly apply a surface-to-core compensation curve: the IR sensor only ever measures surface (skin) temperature, and the firmware nonlinearly boosts that reading toward an estimated core body temperature. This correction is typically small near normal body temperature and deliberately grows steeper as readings climb toward fever range, so that a slightly elevated surface reading gets flagged more aggressively as a likely fever.

A second, related possibility is that the compensation formula takes a second input - most plausibly an ambient temperature reading from a secondary sensor register - which would explain why the same object-temperature raw value produced different displayed results on different occasions. Resolving this is documented as an open item at the end of this write-up.

Phase 5 — Automating the Push Button

Every reading up to this point still required physically pressing the thermometer's button. The next step was to trigger a reading electronically, using the ESP32 itself, controlled by typing a single character into Serial Monitor.

How a button press was simulated

A momentary push button on this kind of PCB almost always has exactly two electrically distinct legs: one tied to ground, and one tied to a GPIO input on the main MCU that is normally held HIGH by a pull-up resistor. Pressing the button momentarily shorts that GPIO to ground, and the MCU's firmware detects the resulting LOW pulse as a button press.

The two legs were identified using a multimeter in continuity mode: one probe held on a known ground point (such as battery negative), the other touched to each button leg in turn. The leg that beeped (showed continuity to ground) was the ground pad; the other was the MCU-side pad.

- The ground pad was wired to the ESP32's own GND pin.
- The MCU-side pad was wired to a spare ESP32 GPIO (GPIO4 in this build), configured in software as a high-impedance INPUT by default - meaning it has zero effect on the button circuit while idle, and the physical button continues to work normally.
- On receiving the character 'S' over serial, the firmware briefly reconfigures that pin to OUTPUT, drives it LOW for about 150 milliseconds (electrically identical to a human pressing the button), then switches it back to INPUT to release it.

Because the pin only ever becomes an output for a fraction of a second and defaults back to a floating input, this technique cannot get the ESP32 and the thermometer's own MCU into a conflict over driving the same line - there is never a moment where both sides are actively driving opposite logic levels.

Phase 6 — Adding WiFi Control

With serial-triggered readings working, the final step was to remove the need for a USB-tethered laptop entirely. The ESP32 was set up to join the local WiFi network on boot and host a small built-in web server, turning any phone or computer on the same network into a remote control for the thermometer.

- On startup, the ESP32 connects to the configured WiFi network and prints its assigned IP address to Serial Monitor.
- Visiting that IP address in a browser loads a minimal page showing the most recent computed temperature, auto-refreshing every two seconds.
- A “Take Reading” button on that page calls the same simulated button-press routine used by the serial 'S' command, so readings can now be triggered from across the room, or from a phone, with no cable attached.

This was implemented with the ESP32's standard WiFi and WebServer libraries, exposing three endpoints: the root page, a /trigger endpoint that fires the simulated press, and a /status endpoint that the page polls to display the latest reading.

Current Status and Open Questions

At this point, the project has:

- A confirmed method for identifying which EEPROM addresses correspond to a live measurement versus a simple counter.
- A working passive I2C sniffer capable of watching any device on the bus without disturbing it.
- A confirmed raw-to-Fahrenheit formula for normal-range body temperature readings, plus detection of the sensor's built-in error sentinel.
- A fully electronic way to trigger a reading, controllable from Serial Monitor or from any browser on the same WiFi network.

The one unresolved piece is the nonlinearity seen at higher (fever-range) readings. The next planned step is to capture every write frame occurring around a single reading - not just the primary 0x26 value - to check whether a second register (most likely an ambient temperature) is also being written and factored into the final compensated result. Once enough high-temperature data points are gathered, the compensation curve can be fit and added to the decoder alongside the existing linear formula.

Summary

What started as a sealed consumer device with no documentation was opened up methodically, one layer at a time: first by impersonating its EEPROM to see what it stored, then by passively listening to its real sensor traffic, then by correlating that raw data against real displayed readings to reverse the conversion formula, and finally by giving the device a fully automated, WiFi-controllable interface. Each phase built directly on evidence gathered in the previous one, turning a black box into a well-understood, remotely operable instrument.